

INTRODUCTION TO RETRIEVAL AUGMENTED GENERATION (RAG)

Yogesh Haribhau Kulkarni

Outline

- 1 INTRODUCTION
- 2 FRAMEWORK
- 3 IMPLEMENTATION
- 4 CONCLUSIONS
- 5 REFERENCES

Retrieval Augmented Generation (RAG)

Motivating Questions

- ▶ **Private Data Access:** How can an LLM-based chatbot answer accurately from data it has *never seen during training*?
- ▶ **Recent Data Access:** How can an LLM-based chatbot answer accurately from data which is recent?
- ▶ **Real-Time Data Access:** How can an LLM-based chatbot answer accurately from continuously changing, real-time data?

These are the core problems RAG is designed to solve.

Need for Domain Adaptation

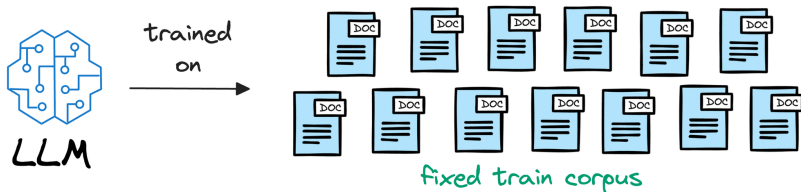
Why can't we just use a general LLM directly?

- ▶ **Privacy & Confidentiality:** Enterprise data (contracts, emails, reports) cannot be sent to public cloud models.
- ▶ **Knowledge Staleness:** LLMs are trained on a fixed snapshot of the web; they know nothing about yesterday's news or your internal data.
- ▶ **Hallucinations:** LLMs confidently fabricate facts when they lack grounding in real documents.
- ▶ **Cost of Retraining:** Training a 7B-parameter model from scratch costs **millions of dollars** and weeks of GPU time.

Key insight: We want the *reasoning ability* of a large LLM, but grounded in *our own up-to-date documents*. Retraining from scratch is not the answer.

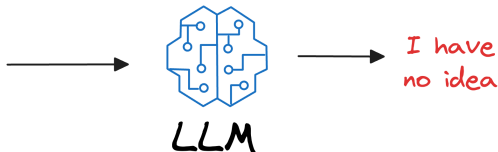
Training and Query Mismatch

LLMs are trained on a static snapshot of the web; real-world queries demand current, private, or domain-specific knowledge that was never in the training data.



Prompt

Election results were
declared on 2nd Feb.
What happened?



Three Approaches to Domain Adaptation

- ▶ **Few-Shot Prompting:** Include 2–5 worked examples directly in the prompt.
Simple but limited by context window; does not scale to large knowledge bases.
- ▶ **RAG (Retrieval-Augmented Generation):** At query time, *fetch* the relevant documents from a database and inject them into the prompt.
No retraining required; works with any LLM; knowledge stays current.
- ▶ **Fine-Tuning (SFT):** Re-train the LLM on domain-specific examples to bake knowledge into its weights.
Expensive; knowledge becomes stale; useful for style/tone adaptation.

RAG is the most practical and cost-effective starting point for most enterprises.

RAG vs SFT (Supervised Fine - Tuning)

RAG & SFT: Complementary Techniques

RAG Features

- ▶ Connects to dynamic external data sources.
- ▶ Reduces hallucinations.
- ▶ Increases transparency in source of information.
- ▶ Works well with very large foundation models.
- ▶ Does not impact style, tone, vocabulary.

SFT Features

- ▶ Changes style, vocabulary, tone of foundation model.
- ▶ Can reduce model size.
- ▶ Useful for deep domain expertise.
- ▶ May not address hallucinations.
- ▶ No improvement in transparency (black box models).

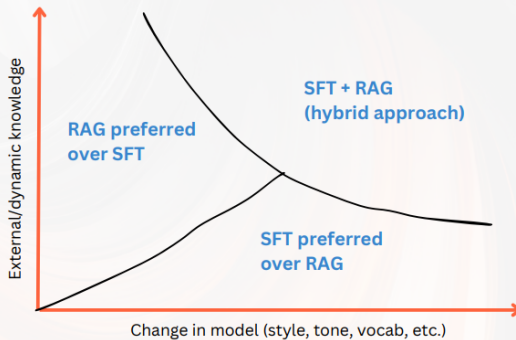
Important Use Case Considerations

Do you require usage of dynamic external data?

RAG preferred over SFT

Do you require changing the writing style, tonality, vocabulary of the model?

SFT preferred over RAG



(Ref: Generative AI with Large Language Model - Abhinav Kimothi)

Other Considerations

- ▶ **Latency:** RAG introduces inherent latency due to search and retrieval.
- ▶ **Scalability:** RAG pipelines are modular and scalable; SFT requires retraining.
- ▶ **Cost:** Both methods require upfront investment; costs vary.
- ▶ **Expertise:** RAG pipelines are moderately simple with frameworks; SFT needs deep understanding and training data creation.

RAG in Plain English: An Analogy

Think of RAG like an “Open-Book Exam”:

- ▶ **Closed-Book LLM:** The model must answer only from what it memorized during training. If the answer is not in its memory, it may guess (hallucinate).
- ▶ **Open-Book RAG:** Before answering, the model *searches* a library of documents, reads the relevant pages, and then constructs a grounded answer.

The three steps of RAG:

1. **Retrieve:** Find the most relevant document chunks from your knowledge base.
2. **Augment:** Inject those chunks into the LLM’s prompt as context.
3. **Generate:** The LLM produces an answer grounded in the retrieved context.

The LLM’s *reasoning* is its own; the *facts* come from your documents.

Why RAG?

▶ **Extended Knowledge:**

- ▶ RAG enables access to external sources, extending beyond the model's training data.
- ▶ Allows querying proprietary documents, recent content, and internet searches.

▶ **Easier to Update/Maintain:**

- ▶ Offers a cost-effective way to update and maintain knowledge in LLMs.
- ▶ Building a knowledge base minimizes ongoing maintenance financial burden.

▶ **Confidence in Responses:**

- ▶ Enhances confidence by providing extra context for more accurate responses.
- ▶ Practical boost to overall intelligence in generating responses.

▶ **Source Citation:**

- ▶ RAG provides access to sources, improving transparency in LLM responses.
- ▶ A step towards building trust in LLM systems.

▶ **Reduced Hallucinations:**

- ▶ RAG-enabled LLMs exhibit reduced creative misfires.
- ▶ Solid foundation of information keeps models focused and grounded.

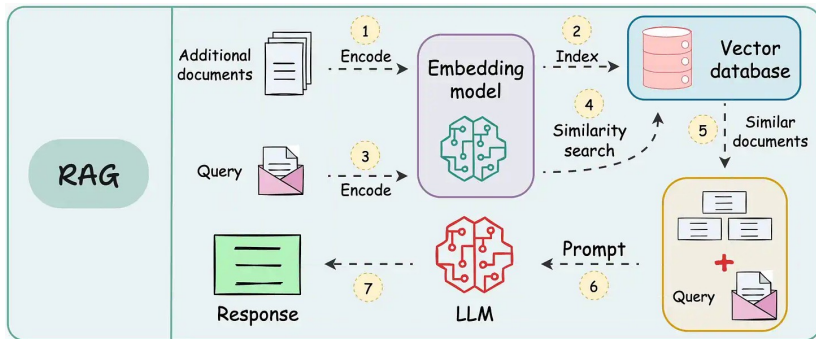
Important Questions

Before adopting RAG or SFT for your use case, answer the following:

- ▶ Does the use case require external data access?
- ▶ Does the use case require changing foundation model style?
- ▶ Does the use case require addressing hallucinations?
- ▶ Is labeled training data available?
- ▶ Is citing the source of information important?
- ▶ How critical is system latency?
- ▶ What are the cost implications?
- ▶ What are the scalability requirements?
- ▶ Do we have the necessary expertise?

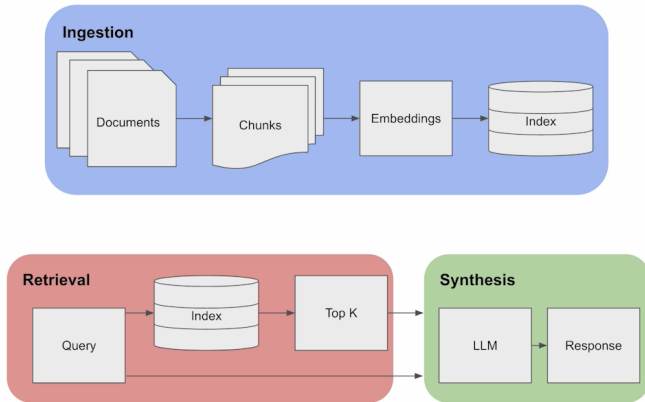
Retrieval Augmented Generation (RAG) Workflow

RAG Workflow



(Ref: A Crash Course on Building RAG Systems -Daily Dose)

RAG Components

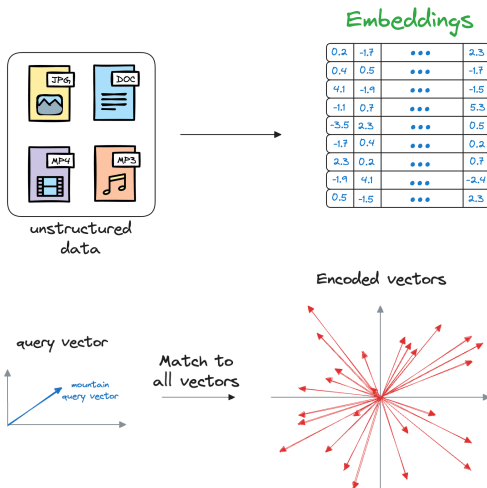


(Ref: [Week 4] Retrieval Augmented Generation -Aishwarya Naresh Reganti)

RAG Components: In Detail

- ▶ Ingestion:
 - ▶ Documents undergo segmentation into chunks, and embeddings are generated from these chunks, subsequently stored in an index.
 - ▶ Chunks are essential for pinpointing the relevant information in response to a given query, resembling a standard retrieval approach.
- ▶ Retrieval:
 - ▶ Leveraging the index of embeddings, the system retrieves the top- k documents when a query is received, based on the similarity of embeddings.
- ▶ Generation:
 - ▶ Examining the chunks as contextual information, the LLM utilizes this knowledge to formulate accurate responses.

Vector Databases



(Ref: A Crash Course on Building RAG Systems -Daily Dose)

Applications

Real-World RAG Applications

- ▶ **Enterprise Q&A:** Chatbots that answer from internal wikis, HR policies, and legal contracts (e.g., Notion AI, Confluence AI).
- ▶ **Customer Support:** Grounded responses from product manuals and FAQs, reducing hallucination risk.
- ▶ **Medical/Legal Research:** Retrieve clauses from case law or clinical trial papers before generating summaries.
- ▶ **Code Assistance:** Retrieve relevant API docs and code snippets, then generate accurate code (e.g., GitHub Copilot).
- ▶ **Personalized Education:** Fetch curriculum-specific notes before answering student questions.

Common theme: Whenever the answer requires *specific, up-to-date, or private* information, RAG is the right tool.

Open Source Resources and Tools

- ▶ **Orchestration Frameworks:** LangChain, LlamaIndex, abstract away chunking, embedding, retrieval, and prompting into pipelines.
- ▶ **Vector Databases:** ChromaDB (local), Weaviate, Qdrant, Pinecone (managed).
- ▶ **Embedding Models:** sentence-transformers from Hugging Face (e.g., all-MiniLM-L6-v2).
- ▶ **Evaluation:** RAGAS framework, measures Faithfulness, Answer Relevancy, Context Precision, and Recall.
- ▶ **Datasets & Benchmarks:** BEIR benchmark for retrieval, TriviaQA, Natural Questions.

RAG Framework

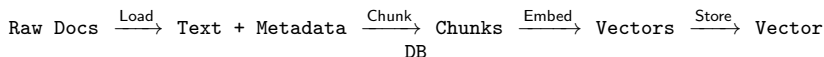
RAG Pipeline: End-to-End View

- ▶ **Ingestion:** Load documents → chunk → embed → store in vector database. Done once; re-run only when knowledge changes.
- ▶ **Retrieval:** Transform user query → embed → search vector DB → rerank top- k chunks.
- ▶ **Generation:** Combine retrieved chunks with the query into a prompt → LLM produces a grounded answer with citations.

Stage 1: Ingestion

Ingestion Overview

Goal: Convert raw, heterogeneous documents into a searchable vector index.



- ▶ **Offline step:** Run once at setup; re-run only when source documents change.
- ▶ **Quality cascade:** Every step feeds the next. Poor chunking $\Rightarrow$ poor embeddings $\Rightarrow$ poor retrieval $\Rightarrow$ poor answers. Fix problems here, not in the LLM.
- ▶ **Key metadata to preserve** at load time: source filename, URL, page number, timestamp: needed later for source citations in generated answers.

Step 1: Loading Data

What can be loaded into a RAG system?

- ▶ Web pages & HTML
- ▶ PDFs, Word (.docx), PowerPoint
- ▶ Source code (Python, Java ...)
- ▶ Structured data: JSON, CSV
- ▶ REST APIs
- ▶ File directories
- ▶ SQL / NoSQL databases
- ▶ Email, Slack, Confluence ...

Frameworks:

- ▶ **LangChain**: DocumentLoader classes for each source; normalises to a Document(page_content, metadata) object.
- ▶ **LlamaIndex**: Reader connectors with the same interface.
- ▶ **Docling** (IBM): structure-aware loader for PDFs and Office documents; preserves tables, headings, and images instead of extracting raw character streams.

Step 2: Chunking: Why

Why must documents be split into chunks?

- ▶ **Context window limit:** LLMs accept at most N tokens per call (4K–1M+ depending on model). A 200-page PDF cannot fit in a single prompt.
- ▶ **Retrieval precision:** The embedding of a 10-page chapter encodes *everything*: the similarity score is diluted. A focused 300-token chunk about one topic scores much higher against a precise query.
- ▶ **Cost:** Fewer tokens per retrieval call = lower inference cost.

The chunking trade-off:

Chunk size	Retrieval precision	Context richness
Too small (<100 tokens)	Very high	Answer may be split across chunks
Balanced (256–512 tokens)	Good	$\approx$ 1 paragraph; general-purpose
Too large (>1500 tokens)	Low	Rich context; diluted similarity score

Overlap: A 10–20% token overlap between adjacent chunks prevents an answer from being cut exactly at a boundary and disappearing from retrieval.

Step 2: Chunking: Strategies

- ▶ **Fixed-size (character split):** Split every N characters. Simple and fast; may cut mid-sentence.
- ▶ **Recursive character split:** Tries paragraph break $\rightarrow$ sentence break $\rightarrow$ word break in order until target size is met. **Recommended default** for generic text (LangChain RecursiveCharacterTextSplitter).
- ▶ **Token-based split:** Counts actual LLM tokens using the target model's tokenizer (e.g., HuggingFace AutoTokenizer). More accurate than character count for models with variable token sizes.
- ▶ **Sentence-aware split:** Splits only at sentence boundaries (spaCy). Preserves full sentences; produces variable chunk sizes.
- ▶ **Semantic split:** Groups consecutive sentences by embedding similarity; creates cohesive topic-based chunks. Best quality; computationally heavier.
- ▶ **Structure-aware split:** Honours document format: Markdown headings (#), HTML tags, code blocks, LaTeX environments. Critical for structured documents.

Practical tip: Always test two or three chunk sizes against a representative query set before committing. Use RAGAS Context Precision as the discriminating metric.

(Ref: Mastering RAG: Advanced Chunking Techniques - Pratik Bhavsar)

Step 3: Embeddings

What is an embedding? An embedding converts a text chunk into a dense vector $\mathbf{v} \in \mathbb{R}^d$ (typically $d = 384\text{--}1536$) such that *semantically similar texts are geometrically close*.

How retrieval uses embeddings:

$$\text{similarity}(q, c_i) = \cos(\mathbf{q}, \mathbf{v}_i) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\| \|\mathbf{v}_i\|}$$

Both the query and every stored chunk are embedded. The top- k chunks with highest cosine similarity to the query are retrieved.

Popular embedding models:

- ▶ **Sentence-BERT / all-MiniLM-L6-v2:** Lightweight, fast, open-source.
- ▶ **E5-large / BGE-large:** Top MTEB scorers; open-source.
- ▶ **OpenAI text-embedding-3-small:** Strong general-purpose; API-based.
- ▶ **word2vec / GloVe / fastText:** Word-level; context-free; mostly superseded.
- ▶ **Domain fine-tuned:** BioBERT (medical), LegalBERT (legal): win in specialist domains.

Choosing an Embedding Model

No universal answer. Use the **MTEB Leaderboard** (Massive Text Embedding Benchmark, Hugging Face) as the starting point. It benchmarks models across 7 tasks including Retrieval, Reranking, and Semantic Similarity.

Decision factors:

- ▶ **Accuracy vs. speed:** Large models score higher on MTEB but are slower. `all-MiniLM-L6-v2` is $5\times$ faster than large BERT with modest accuracy loss.
- ▶ **Domain:** General models work for most cases; fine-tune or use a domain model for specialised vocabulary (medical, legal, code).
- ▶ **Cost:** API-based models (OpenAI, Cohere) charge per token. Open-source models run free locally.
- ▶ **Multilingual:** Use `multilingual-e5` or Cohere multilingual for non-English documents.
- ▶ **Max input length:** Each model has a token limit (e.g., 512 tokens for MiniLM). Chunks longer than this are truncated, losing information.

Creating embeddings: `LangChain OpenAIEmbeddings / HuggingFaceEmbeddings; LlamaIndex resolve_embed_model.`

(Ref: <https://huggingface.co/spaces/mteb/leaderboard>)

Step 4: Vector Databases

What is a vector database? A specialised datastore for high-dimensional vectors. It enables fast *approximate nearest-neighbour (ANN)* search, plus features a plain in-memory index (FAISS) cannot provide: persistent storage, metadata filtering, access control, and scalability.

Database	Type	Best for	Notes
FAISS	Library	Local proto-typing	No persistence; no metadata
ChromaDB	Open-source	Local dev / testing	Easy LangChain integration
Qdrant	Open-source	Production OSS	Rich filtering; on-prem or cloud
Weaviate	Open-source	Hybrid search	Keyword + vector built-in
Pinecone	Managed SaaS	Large-scale cloud	Fully managed; pay-per-use
pgvector	PG extension	Existing Postgres	SQL + vector in one DB

Choosing: Prototype with ChromaDB or FAISS. For production, weigh: accuracy vs. query speed, local vs. cloud, metadata filtering needs, and operational cost.

Storing embeddings (LangChain example):

Stage 2: Retrieval

Retrieval Overview

Goal: Given a user query, return the k most relevant chunks from the vector store.

User Query $\xrightarrow{\text{Transform}}$ Better Query/Queries $\xrightarrow{\text{Search}}$ Candidates $\xrightarrow{\text{Rerank} + \text{Filter}}$ Top- k Chunks

- **Why transform the query first?** Raw user queries are often short, ambiguous, or use different terminology than the indexed documents. Transforming them before search dramatically improves recall.
- **Why rerank after search?** Embedding similarity is a coarse, approximate signal. A cross-encoder reranker scores query–chunk pairs *jointly* and produces a much more accurate relevance ordering.

Query Transformations

Rewriting or expanding the query before search aligns it better with indexed document language.

- ▶ **Multi-query Expansion:** LLM generates 3–5 alternative phrasings of the same question. Each is searched independently; results are union-merged and deduplicated. *Higher recall: catches documents that match some phrasings but not others.*
- ▶ **Query Rewriting:** LLM rephrases a poorly formed query into a clear, search-friendly statement. Fixes typos, jargon, or ambiguous pronouns.
- ▶ **Query Compression:** In a multi-turn conversation, the full chat history is condensed into one standalone question before retrieval. Removes noise from earlier turns.
- ▶ **HyDE (Hypothetical Document Embedding):** LLM generates a plausible fake answer to the query; that answer is embedded and used as the search vector instead of the raw query. Answer-to-answer similarity is stronger than short query-to-document similarity.
- ▶ **Query Routing:** A classifier directs the query to the most appropriate data source (vector DB, SQL database, web search, calculator) based on detected intent.

(Ref: <https://arxiv.org/pdf/2212.10496.pdf>)

Retrieval Methods

- ▶ **Dense (Similarity) Search:** Embed query → cosine/dot-product search over all chunk vectors. Default in most RAG systems.
- ▶ **Sparse (Keyword) Search:** BM25 (Best Match 25, a TF-IDF-based ranking function) over token frequencies. Excellent for exact-match terms: product codes, proper nouns, that dense search misses.
- ▶ **Hybrid Search:** Combines dense + sparse scores (e.g., Reciprocal Rank Fusion). Semantic understanding *plus* lexical precision. Generally outperforms either alone.
- ▶ **Maximum Marginal Relevance (MMR):** Selects chunks that are relevant *and* diverse. Iteratively picks the next chunk that maximises relevance minus overlap with already-selected chunks. Reduces redundancy in retrieved context.
- ▶ **Contextual Compression:** Retrieved chunks are passed through an LLM extractor that returns only the sentences most relevant to the query, reducing noise before generation.
- ▶ **Time-weighted Retrieval:** Combines semantic score with a recency bonus so fresher documents are preferred when similarity scores are close.

Reranking & Post-Retrieval Filtering

Why rerank? First-pass vector search scores query and chunk *independently*. A cross-encoder scores the (query, chunk) *pair jointly*, giving far higher accuracy: but is too slow to run over the full corpus, so it runs only on the top- k candidates.

$$s_i = \text{CrossEncoder}([\text{query}; \text{chunk}_i])$$

Sentence Window Retrieval: Index at fine sentence granularity for precise matching, but return the surrounding $\pm N$ sentence window to the LLM for fuller context.

Post-retrieval filtering options:

- ▶ **Score threshold:** Drop chunks below a minimum relevance score.
- ▶ **Metadata filter:** Keep only chunks from a specific date range, document type, or author.
- ▶ **Deduplication:** Remove near-identical chunks before passing to the LLM to avoid redundancy.
- ▶ **Auto-merging:** If $>50\%$ of a parent document's child chunks are retrieved, replace them with the full parent chunk for richer context.

(Ref: <https://www.pinecone.io/learn/series/rag/rerankers/>)

Retrieval: End-to-End Example (LangChain)

Full pipeline: load → chunk → embed → store → retrieve

1. Load text with TextLoader
2. Split:
RecursiveCharacterTextSplitter
chunk=512, overlap=50
3. Embed: all-MiniLM-L6-v2
4. Store in Chroma vector DB
5. Retrieve: similarity_search, $k = 4$

```
1 from langchain_community.document_loaders import TextLoader
2
3 from langchain.text_splitters import (
4     RecursiveCharacterTextSplitter
5 )
6 from langchain_community.embeddings import (
7     HuggingFaceEmbeddings
8 )
9 from langchain_community.vectorstores import Chroma
10
11 docs = TextLoader("manual.txt").load()
12
13 chunks = RecursiveCharacterTextSplitter(
14     chunk_size=512, chunk_overlap=50
15 ).split_documents(docs)
16
17 embed = HuggingFaceEmbeddings(
18     model_name="all-MiniLM-L6-v2"
19 )
20 db = Chroma.from_documents(chunks, embed)
21
22 results = db.similarity_search(
23     "What is the return policy?", k=4
24 )
```

Stage 3: Generation

Generation: Prompt Construction

The LLM sees exactly what you put in the prompt. Prompt quality determines answer quality.

Standard RAG prompt structure:

1. **System instruction:** Sets role and grounding rules.
Example: "Answer only from the provided context. If the answer is not in the context, say 'I don't know.' Do not add external knowledge."
2. **Retrieved context:** The top- k chunks, each labelled with its source file and page.
3. **User query:** The original question.

Context window budget (example for a 4K-token model):

Component	Typical tokens	Budget %
System prompt	100–300	5–8%
Retrieved chunks	1500–2500	60–70%
User query	20–100	<3%
Reserved for answer	300–800	15–20%

Lost-in-the-middle: LLMs attend less to chunks in the middle of long contexts. Place the most relevant chunk *first or last* in the context block.

Generation: Response Synthesis Strategies

How should the LLM combine multiple retrieved chunks into one answer?

- ▶ **Stuffing (default):** Concatenate all chunks into one prompt and generate one answer in a single LLM call. Simple and fast; fails when total tokens exceed the context window.
- ▶ **Map-Reduce:** Process each chunk separately with the LLM to produce partial answers (map); then merge all partial answers in a second LLM call (reduce). Handles large corpora; more expensive.
- ▶ **Summarise-then-Generate:** Summarise the retrieved context first; pass the condensed summary to the LLM. Reduces token cost; risks losing fine-grained details.
- ▶ **Iterative Refinement:** Start with an answer from chunk 1; pass it together with chunk 2 to the LLM for refinement; repeat. Final answer incorporates all evidence. Higher quality; sequential latency.

Fine-tuning the generation layer (if base quality is still insufficient):

- ▶ **Encoder fine-tuning:** Improve embedding quality for the domain.
- ▶ **RA-DIT** (Retrieval-Augmented Dual Instruction Tuning, Lin et al. 2023): joint fine-tuning of the LLM and retriever on (query, context, answer) triplets.

Stage 4: Evaluation

What to Evaluate and Why

A RAG pipeline has two independently failure-prone components. Evaluate them separately.

Retrieval quality

- ▶ Did the retriever fetch the right chunks?
- ▶ How much noise is in the top- k ?
- ▶ Did it miss any required documents?

Fix: chunking, embedding model, query transformation, reranking

Generation quality

- ▶ Is the answer grounded in the context?
- ▶ Does it actually answer the question?
- ▶ Does it match the known correct answer?

Fix: system prompt, LLM choice, synthesis strategy

Evaluation data required:

- ▶ Test **queries** representative of real user questions.
- ▶ **LLM response** for each query.
- ▶ **Retrieved context** used for each query.
- ▶ (For labelled metrics) A **ground truth** answer per query.

RAGAS: Evaluation Framework

RAGAS (Retrieval Augmented Generation Assessment) is an open-source framework that evaluates a RAG pipeline *without needing human annotations for most metrics*: it uses an LLM as the judge. It covers both retrieval quality and generation quality with a single `evaluate()` call.

Faithfulness consistency of the answer with the context (but no query!) Two LLM calls, get context that was used to derive to answer, check if the statement supported by the context	Context Relevance how is the retrieved context “focused” on the answer , the amount of relevant info vs noise info , uses LLM to compute relevance of sentences / total number of retrieved sentences
Answer Relevancy is the answer relevant to the query , LLM call, get queries that may generate answer, verify if they are similar to the original query	Context Recall (ext, optional) if all relevant sentences are retrieved , assuming existence of ground_truth answer

(Ref: Evaluating LLM Models for Production Systems: Methods and Practices - Andrei Lopatenko)

RAGAS Metrics: Summary

Metric	What it measures	Fix if low
Faithfulness	Hallucination	Grounding prompt
Answer Relevance	On-topic response	Query / prompt
Context Recall	Retriever coverage	$\uparrow k$, re-embed
Context Precision	Retriever noise	Rerank, filter
Context Relevance	Context signal/noise	Rerank, filter
Answer Correctness	End-to-end quality	Full pipeline

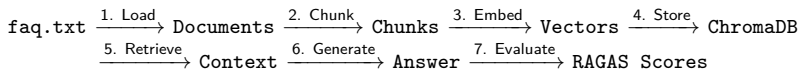
No-label quick start: Run Faithfulness + Answer Relevancy + Context Relevancy continuously in production.

Labelled offline eval: Add Context Recall, Context Precision, Answer Correctness on a curated test set.

RAG Pipeline: End-to-End Implementation

Implementation Overview

Goal: Build a question-answering system over a local `faq.txt` file and measure its quality with RAGAS: using only open-source components.



The Data: `faq.txt`

- ▶ A plain-text FAQ file: the simplest possible real-world knowledge source.
- ▶ Each entry follows a **Q:** / **A:** format; entries are separated by a blank line.
- ▶ Plain text is ideal for an academic demo: no PDF parsing complexity, no OCR errors, fully reproducible.
- ▶ The same pattern scales to any `.txt`, `.md`, or `.rst` knowledge base.
- ▶ `TextLoader` will read the entire file into a single `LangChain Document` object; the splitter then breaks it into per-entry chunks.

```
1 # faq.txt (excerpt)
2
3 Q: What is RAG?
4 A: Retrieval—Augmented Generation (RAG)
5   combines a retrieval system with a
6   language model. The retriever fetches
7   relevant documents; the LLM generates
8   an answer grounded in those documents.
9
10 Q: Why use RAG instead of fine—tuning?
11 A: RAG keeps knowledge outside the model
12   weights, so it can be updated without
13   retraining. Fine—tuning is expensive
14   and risks forgetting prior knowledge.
15
16 Q: What is a vector database?
17 A: A database optimised for storing and
18   searching high—dimensional embedding
19   vectors using approximate nearest—
20   neighbour (ANN) algorithms.
21
22 Q: What is chunking?
23 A: Splitting documents into smaller pieces
24   so each piece fits in the LLM context
   window and carries a focused meaning.
```


Dependencies and Setup

- ▶ **langchain** ≥ 0.2 : the community-stable API used throughout.
- ▶ **langchain-community**: integrations (loaders, LLMs, vector stores).
- ▶ **langchain-text-splitters**: text splitting utilities (new package in 0.2+).
- ▶ **sentence-transformers**: local embedding model; no API key needed.
- ▶ **chromadb** ≥ 0.5 : lightweight local vector store; auto-persists to disk.
- ▶ **huggingface.hub**: access to open-source LLMs via Inference API.
- ▶ **ragas**: reference-free RAG evaluation framework.
- ▶ **datasets**: required by RAGAS to build the evaluation dataset object.
- ▶ Set `HUGGINGFACEHUB_API_TOKEN` in your

```
1 # Create a virtual environment (recommended)
   conda create -n rag-demo python=3.11
3   conda activate rag-demo

5 # Install dependencies
   pip install langchain==0.2.16
7   pip install langchain-community==0.2.17
   pip install langchain-text-splitters==0.2.4
9   pip install sentence-transformers==3.0.1
   pip install chromadb==0.5.18
11  pip install huggingface.hub==0.24.7
   pip install ragas==0.1.21
13  pip install datasets==2.20.0

15 # Set your HuggingFace token
   export HUGGINGFACEHUB_API_TOKEN="hf..."
```

Step 1: Load and Parse with TextLoader

- ▶ **TextLoader** reads a plain-text file and returns a list of LangChain Document objects, each carrying `page_content` and `metadata`.
- ▶ The file is loaded as a **single Document**; the text splitter in Step 2 divides it into individual Q&A chunks.
- ▶ Each Document automatically receives source metadata (the file path), useful for citations in the final answer.
- ▶ Printing the first document confirms the content was loaded correctly before proceeding.

(Ref: https://python.langchain.com/docs/integrations/document_loaders/)

```
2 # T1d: Updated import namespace — TextLoader
3 # moved to langchain_community in LangChain
4 0.2+
5 from langchain_community.document_loaders \
6     import TextLoader
7
8 loader = TextLoader(
9     file_path="faq.txt",
10    encoding="utf-8"
11 )
12 documents = loader.load()
13
14 print(f"Loaded {len(documents)} document(s)")
15 print("----- First document:")
16 print(documents[0].page_content[:300])
17 print("Metadata:", documents[0].metadata)
18
19 # Expected output:
20 # Loaded 1 document(s)
21 # First document:
22 # Q: What is RAG?
23 # A: Retrieval-Augmented Generation ...
24 # Metadata: {'source': 'faq.txt'}
```

Step 2: Chunking the Document

- ▶ **RecursiveCharacterTextSplitter** is the recommended default for plain text: it tries paragraph, then sentence, then word boundaries before falling back to characters.
- ▶ `chunk_size=300`: each chunk covers roughly one FAQ entry (question + answer).
- ▶ `chunk_overlap=50`: a 50-character tail is repeated in the next chunk, preventing an answer from being split across a boundary.
- ▶ `separators` list prioritises blank lines (`\n\n`) so each Q&A pair stays together.
- ▶ Inspect chunk count and the first chunk to verify boundaries are sensible.

(Ref: https://python.langchain.com/docs/how_to/recursive_text_splitter/)

```
# T1e: Updated import —  
RecursiveCharacterTextSplitter  
2 # moved to langchain-text-splitters package in  
0.2+  
from langchain_text_splitters import \br/>4 RecursiveCharacterTextSplitter  
  
6 splitter = RecursiveCharacterTextSplitter(  
    chunk_size=300,  
    chunk_overlap=50,  
    separators=["\n\n", "\n", " ", ""]  
10 )  
  
12 chunks = splitter.split_documents(documents)  
  
14 print(f"Total chunks: {len(chunks)}")  
print("\n--- Chunk 0:")  
16 print(chunks[0].page.content)  
print("\n--- Chunk 1:")  
18 print(chunks[1].page.content)  
  
20 # Each chunk should contain one Q&A pair.  
# chunk_overlap ensures no answer is lost  
22 # at a boundary.
```

Step 3: Embed and Store in ChromaDB

- ▶ **HuggingFaceEmbeddings** wraps the sentence-transformers library. all-MiniLM-L6-v2 maps text to 384-dimensional vectors: lightweight and fast on CPU.
- ▶ **Chroma.from_documents()** embeds every chunk and stores vectors plus original text on disk under ./chroma_db.
- ▶ `persist_directory` enables loading the index in future sessions without re-embedding.
- ▶ Printing the collection count confirms all chunks were indexed successfully.

(Ref: <https://python.langchain.com/docs/integrations/vectorstores/chroma/>)

```
2 from langchain.community.embeddings import \
   HuggingFaceEmbeddings
3 from langchain.community.vectorstores import
   Chroma
4
5 # Local embedding model — no API key needed
6 embedding_model = HuggingFaceEmbeddings(
7     model_name="all-MiniLM-L6-v2",
8     model_kwargs={"device": "cpu"}
9 )
10
11 # Build and persist the vector store
12 vectorstore = Chroma.from_documents(
13     documents=chunks,
14     embedding=embedding_model,
15     persist_directory="./chroma_db"
16 )
17 # T1f: vectorstore.persist() is DEPRECATED in
18 # ChromaDB >= 0.4.0. Persistence is now
   automatic
19 # when persist_directory is set. Remove this call
20 # in newer installations.
21 # vectorstore.persist()
22
23 count = vectorstore._collection.count()
24 print(f"Indexed {count} chunks in ChromaDB")
```

Step 4: Retriever and Prompt Template

- ▶ `as_retriever(search_kwargs={"k": 3})` returns a `VectorStoreRetriever` that runs cosine similarity search and returns the top-3 chunks.
- ▶ The **PromptTemplate** injects both `{context}` (retrieved chunks) and `{question}` (user query) into a single formatted string.
- ▶ The grounding instruction: *"use only the provided context"*: is the key safeguard against hallucination. If the answer is absent from context, the model is instructed to say so.
- ▶ `input_variables` must match exactly the placeholder names used in the template string.

```
2 from langchain.prompts import PromptTemplate
3
4 # Retriever: return top-3 similar chunks
5 retriever = vectorstore.as_retriever(
6     search_type="similarity",
7     search_kwargs={"k": 3}
8 )
9
10 # Grounded prompt template
11 template = """You are a helpful assistant.
12 Answer the question using ONLY the context
13 provided below. If the answer is not in
14 the context, say "I don't know."
15
16 Context:
17 {context}
18
19 Question: {question}
20
21 Answer: """
22
23 prompt = PromptTemplate(
24     input_variables=["context", "question"],
25     template=template
26 )
```

Step 5: LLM and RetrievalQA Chain

- ▶ **HuggingFaceHub** calls the HuggingFace Inference API: no local GPU needed. `google/flan-t5-large` is a compact instruction-following model well-suited to Q&A.
- ▶ `max_new_tokens=256` caps response length; `temperature=0` makes output deterministic: important for reproducible academic experiments.
- ▶ **RetrievalQA.from_chain_type()** wires the retriever, prompt, and LLM into a single callable object.
- ▶ `return_source_documents=True` exposes which chunks were used, enabling manual inspection and RAGAS evaluation.

(Ref: <https://python.langchain.com/docs/integrations/llms/>)

```
1 # T1g: Updated import namespace —  
   HuggingFaceHub  
2 # moved to langchain.community.llms in  
   LangChain 0.2+  
3 from langchain_community.llms import  
   HuggingFaceHub  
4 from langchain.chains import RetrievalQA  
5  
6 # Open-source LLM via HuggingFace Inference  
   API  
7 llm = HuggingFaceHub(  
8     repo_id="google/flan-t5-large",  
9     model_kwargs={  
10         "temperature": 0,  
11         "max_new_tokens": 256  
12     }  
13 )  
14  
15 # Wire retriever + prompt + LLM into one chain  
qa_chain = RetrievalQA.from_chain_type(  
16     llm=llm,  
17     chain_type="stuff",  
18     retriever=retriever,  
19     return_source_documents=True,  
20     chain_type_kwargs={"prompt": prompt}  
21 )
```

Step 6: Query the Pipeline

- ▶ Calling `qa_chain()` with a query string triggers the full pipeline in one step: embed query → retrieve top-3 chunks → format prompt → LLM generates answer.
- ▶ The result dict contains "result" (the answer string) and "source_documents" (the retrieved Document objects).
- ▶ Printing source documents lets you verify the retriever fetched relevant chunks: the first diagnostic step when answers look wrong.
- ▶ Inspect `page_content` of each source doc to confirm it contains evidence for the answer.

```
1 query = "What is RAG and why is it useful?"
2
3
4 result = qa_chain({"query": query})
5
6 print("Question:", query)
7 print("\nAnswer:", result["result"])
8 print("\n---- Retrieved chunks:")
9 for i, doc in enumerate(
10     result["source_documents"], 1):
11     print(f"\n[Chunk {i}]")
12     print(doc.page_content[:200])
13
14 # Sample output:
15 # Answer: RAG combines a retrieval system
16 # with a language model. It is useful
17 # because knowledge can be updated without
18 # retraining the model weights.
19 #
20 # [Chunk 1]
21 # Q: What is RAG?
22 # A: Retrieval-Augmented Generation ...
```

Step 7: RAGAS Evaluation Dataset

- ▶ RAGAS requires four parallel lists for evaluation:
 - ▶ **questions**: the test queries.
 - ▶ **answers**: LLM responses from our pipeline.
 - ▶ **contexts**: the retrieved chunk texts for each query (list of lists).
 - ▶ **ground_truths**: known correct answers (list of lists); needed for Recall and Correctness metrics.
- ▶ Run the pipeline for each test question in a loop; collect the answer and source document texts.
- ▶ Wrap everything in a HuggingFace Dataset object: the format RAGAS evaluate() expects.

(Ref: <https://docs.ragas.io/en/stable/getstarted/evaluation/>)

```
1 from datasets import Dataset
3 test_questions = [
4     "What is RAG?",
5     "Why use RAG instead of fine-tuning?",
6     "What is a vector database?",
7     "What is chunking in RAG?"
8 ]
9 ground_truths = [
10     ["RAG combines retrieval with an LLM."],
11     ["RAG updates knowledge without retraining."],
12     ["A DB for storing embedding vectors."],
13     ["Splitting documents into smaller pieces."],
14 ]
15
16 answers, contexts = [], []
17 for q in test_questions:
18     res = qa_chain({"query": q})
19     answers.append(res["result"])
20     contexts.append([
21         d.page_content
22         for d in res["source_documents"]
23     ])
24
25 eval_dataset = Dataset.from_dict({
26     "question": test_questions,
27     "answer": answers,
28     "contexts": contexts,
29     "ground_truths": ground_truths
30 })
```


Step 8: RAGAS Metrics and Results

- ▶ **faithfulness**: are all claims in the answer verifiable from retrieved context? Scores below 0.7 indicate hallucination.
- ▶ **answer_relevancy**: does the answer address the question? Low score means the response is off-topic.
- ▶ **context_recall**: did the retriever fetch all evidence present in the ground truth?
- ▶ **context_precision**: what fraction of retrieved chunks was actually useful?
- ▶ All scores are in $[0, 1]$; higher is better. For a clean FAQ, scores above 0.85 are achievable with default settings.

(Ref: <https://docs.ragas.io/en/stable/concepts/metrics/>)

```
1 from ragas import evaluate
2 from ragas.metrics import (
3     faithfulness,
4     answer_relevancy,
5     context_recall,
6     context_precision
7 )
8
9 # RAGAS uses an LLM internally as judge.
10 # Default judge model: gpt-3.5-turbo.
11 # For a fully local setup use a local LLM:
12 # from ragas.llms import
13     LangchainLLMWrapper
14 # ragas.llm = LangchainLLMWrapper(llm)
15
16 result = evaluate(
17     dataset=eval_dataset,
18     metrics=[
19         faithfulness,
20         answer_relevancy,
21         context_recall,
22         context_precision
23     ]
24 )
25
26 # Print scores as a pandas DataFrame
27 df = result.to_pandas()
28 print(df[["question",
29         "faithfulness",
30         "answer_relevancy",
31         "context_recall",
32         "context_precision"]])
```

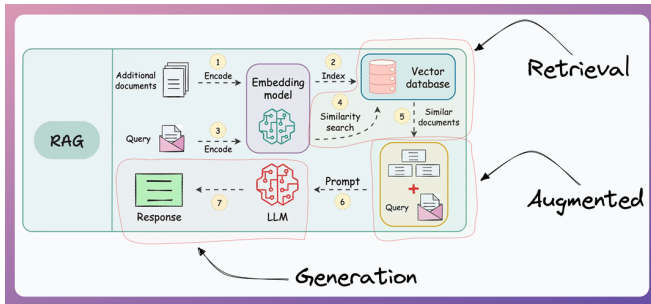
Key design decisions and their rationale

- ▶ `chunk_size=300, k=3`: Each FAQ entry fits in one chunk; 3 chunks give enough context without overflowing the prompt.
- ▶ **Grounding instruction in prompt**: The single highest-impact step to reduce hallucination: forces the LLM to stay within retrieved evidence.
- ▶ `temperature=0`: Makes every run deterministic; essential for reproducible academic experiments.
- ▶ **RAGAS without ground truth**: *faithfulness* and *answer_relevancy* need no labels: run them in production for continuous monitoring.
- ▶ **Scaling up**: Replace `faq.txt` with any document set; swap Chroma for Qdrant or Pinecone; replace `flan-t5-large` with any LangChain-compatible LLM.
- ▶ **Modern alternative (LCEL)**: LangChain 0.2+ recommends the LangChain Expression Language (`|` operator) over `RetrievalQA` for composable, type-safe chains: `chain = prompt | llm | StrOutputParser()`.

Conclusions

So, What is RAG?

- ▶ RAG grounds LLM responses in **retrieved, verifiable documents**: the LLM reasons; external documents supply the facts.
- ▶ **Motivation**: Overcome knowledge cutoff dates, hallucinations, and lack of access to private or real-time data.
- ▶ **Impact**: State-of-the-art performance on knowledge-intensive tasks: enterprise Q&A, document search, code assistance, and medical/legal research.



(Ref: A Crash Course on Building RAG Systems -Daily Dose)

RAG Architecture: The Two Pipelines

► Indexing Pipeline (offline):

1. Load documents from diverse sources.
2. Split into chunks (chunk size: 256–512 tokens typical).
3. Embed each chunk $\rightarrow$ dense vector $\mathbf{v} \in \mathbb{R}^d$.
4. Store $(\mathbf{v}, \text{chunk text, metadata})$ in a vector database.

► Query Pipeline (online, per user request):

1. Embed user query $\mathbf{q} \in \mathbb{R}^d$.
2. Find top- k chunks: $\arg \max_i \cos(\mathbf{q}, \mathbf{v}_i)$.
3. Build prompt: [System] + [Retrieved chunks] + [User query].
4. LLM generates a grounded answer.

► Evaluation Pipeline (optional): Assess Faithfulness and Context Precision using RAGAS on a test set.

(Ref: RAG Architecture – Abhinav Kimothi)

The Core RAG Bottleneck

- ▶ Industry consensus suggests 90% of RAG quality issues stem from poor retrieval—not the LLM.
- ▶ Better LLMs do not fix bad retrieval.
- ▶ Naive RAG: chunk, embed, retrieve top- k — too simplistic for production.
- ▶ Poor retrieval causes hallucinations, omissions, and vague outputs.
- ▶ Quality of generation depends on quality of retrieved context.

RAG Best-Practice Checklist

1. **Fix the Basics First:** Dynamic chunking that respects document structure; tune chunk size; metadata filtering; hybrid search (vector + keyword).
2. **Improve Retrieval:** Re-ranking (learned or rule-based); small-to-big retrieval; multi-hop and agentic retrieval for complex queries.
3. **Evaluate Systematically:** End-to-end RAGAS metrics; component-level MRR (Mean Reciprocal Rank), NDCG (Normalized Discounted Cumulative Gain), and success@ k . Don't iterate blindly.
4. **Fine-Tune as a Last Resort:** Only if retrieval and prompting are fully optimised and the domain is highly specialised. Be aware of added cost, latency, and complexity.

Iterate in this order. 90% of quality gains come from steps 1–3.

Pros and Cons of RAG

Pros:

- ▶ **No retraining required:** plug any LLM into the pipeline.
- ▶ **Grounded responses:** answer is traceable to source documents.
- ▶ **Knowledge stays current:** update the vector store, not the model.
- ▶ **Lower hallucination rate** than a standalone LLM on domain-specific queries.
- ▶ **Source attribution:** users can verify where the answer came from.

Cons:

- ▶ **Added latency:** retrieval + LLM call is slower than LLM alone.
- ▶ **Retrieval can fail:** bad chunking or embeddings means the right context is never found.
- ▶ **Context window limits:** only a finite number of chunks can be injected.
- ▶ **Garbage-in, garbage-out:** quality of answers depends on quality of the document corpus.

Agentic RAG: Beyond One-Shot Retrieval

- ▶ **Limitation of naive RAG:** A single retrieve-then-generate step fails on complex, multi-hop questions that require combining information from several sources.
- ▶ **Agentic RAG:** An LLM agent decides *whether* to retrieve, *what* to retrieve, and *when to stop*: iterating until the answer is sufficiently grounded.
- ▶ **Key patterns:**
 - ▶ **Self-RAG:** The LLM scores its own retrieval need and answer quality at each step; skips retrieval when unnecessary.
 - ▶ **CRAG (Corrective RAG):** Detects when retrieved context is irrelevant and triggers a corrective web search.
 - ▶ **ReAct + RAG:** Interleaves reasoning steps (*Thought:*) with retrieval actions (*Search:*) in a loop.
- ▶ **Frameworks:** LangGraph, LlamaIndex Agents, OpenAI Assistants API.

Trade-off: Higher answer quality on complex queries; variable latency and cost due to multi-step LLM calls.

Challenges and Future Directions

- ▶ **Multi-modal RAG:** Extend retrieval to images, tables, audio, and video (using CLIP-style embeddings or captioning models).
- ▶ **Long-Context RAG:** As LLMs support 1M+ token windows, the trade-off between retrieval and direct context injection will shift.
- ▶ **Self-Correcting RAG:** Systems that can detect their own retrieval failures and automatically re-query (Self-RAG, CRAG).
- ▶ **Graph RAG:** Using knowledge graphs instead of flat vector stores to capture entity relationships and improve multi-hop reasoning.
- ▶ **Privacy-Preserving RAG:** Federated retrieval across siloed enterprise systems without data leakage.
- ▶ **Bias Mitigation:** Ensuring retrieved documents do not introduce selection bias into generated answers.

References

Many publicly available resources have been refereed for making this presentation. Some of the notable ones are:

- ▶ Retrieval Augmented Generation — Medium blogs by Abhinav Kimothi
- ▶ Mastering RAG: How To Architect An Enterprise RAG System — Pratik Bhavsar
- ▶ Prompt Engineering Guide <https://www.promptingguide.ai/techniques/rag>
- ▶ 3 Day RAG Roadmap: Understanding, Building and Evaluating RAG Systems 2024 — Aishwarya Naresh Reganti
- ▶ Explanation of RAG by DeepLearning AI <https://www.deeplearning.ai/short-courses/>
- ▶ What is RAG by DataStax <https://www.datastax.com/guides/what-is-retrieval-augmented-generation>
- ▶ Advanced RAG series (6 videos) by Sam Witteveen
- ▶ LangChain101: Question A 300 Page Book (w/ OpenAI + Pinecone) by Greg Kamradt
- ▶ Blog on advanced RAG techniques by Akash
- ▶ RAG hands-on tutorials on GitHub <https://github.com/gkamradt/langchain-tutorials>
- ▶ RAGAS evaluation framework <https://docs.ragas.io>
- ▶ MTEB Leaderboard (embedding model comparison) <https://huggingface.co/spaces/mteb/leaderboard>

Thanks ...

- ▶ Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- ▶ Call + 9 1 9 8 9 0 2 5 1 4 0 6



(<https://www.linkedin.com/in/yogeshkulkarni/>)



(<https://medium.com/@yogeshharibhaukulkarni>)



(<https://www.github.com/yogeshhk/>)